

ST.ELLA Gold Pipeline

Live Session Quality Audit

Based on a real, end-to-end pipeline run captured via `/api/debug/session-dump` on the deployed Render service, plus direct inspection of the engine source (full pipeline/*.py, gold_engine_commands_full_v1_4_13.json). Prepared 2026-07-20.

Summary of findings

- **Task Response relevance scoring is structurally broken** — an essay can be 100% off-topic and still score TR band 5-6, because the code that is supposed to check topic relevance never actually compares the essay to the assigned prompt. Confirmed at the exact source lines. **Highest priority.**
- **Canonical lexical resources are still not loading in the live container** — LRET's own QA log shows it looked for them at the old, pre-fix path. This degrades every LRET suggestion (fragments, weak/no collocation grounding) and is likely the single biggest driver of the LRET quality complaints.
- **A real FIX/KEEP contradiction bug exists in LRET** — the same span can be flagged as an error to fix and simultaneously praised as good usage. The code has a deduplication step meant to prevent exactly this, and it has a coverage-detection gap.
- **The progress tracker is not actually empty** — real, well-formed data exists in the backend for this session. The "empty" experience is very likely a frontend display bug rather than a missing-data bug.
- **Writing Coach's chosen mission doesn't match the Directive engine's top-priority skill** — expected behavior for a first session (only one grammar error has ever been logged for this student), but the two engines are not reading from a shared "priority skill" signal, which is why it looks arbitrary.

Finding 1 — Task Response relevance check is a no-op

SEVERITY: HIGH — scores can't be trusted for off-topic essays

The essay you submitted was written about politicians' right to privacy. The assigned prompt was about free university education. These share no topic overlap. The pipeline nonetheless returned:

```
task_response (raw): 6
task_response (official band): 5
task_schema_status: "complete"
relevance_ratio (TR4): 0.9
task_resolution_state: "TASK_OK"
```

A real IELTS examiner would score Task Response at band 1-2 for an essay that does not address the task at all. Band 5 is a severe scoring failure, and it is systematic, not a one-off fluke — here is why.

Root cause, with exact source location

TR4_relevance_ratio (the metric that is supposed to measure whether the essay is relevant to the prompt) is built in `evaluator_rubric_bridge_v1.py` from two Evaluator sub-skills:

```
"TR4_relevance_ratio": _avg(
    _cv(sop, "arg_claim_relevance", "relevance"),
    _cv(sop, "maintain_task_focus", "focus"),
)
```

Both of those sub-skills are computed inside `va_premium_evaluator_v8_3_wke_standalone.py`. Tracing them to their actual implementation:

```
# line 1773-1775
if skill_id in ("arg_claim_relevance",):
    # Claim relevant to topic – almost always true if claim exists
    return DEPTH_2 if claims else DEPTH_0
```

The comment in the code admits the bug: this check does not look at the prompt at all. It only asks whether the essay contained any argumentative claim, about anything. If yes, full marks for relevance.

`maintain_task_focus` falls through to the generic dispatcher and has the same problem:

```
# line 1898-1902 (_depth_generic, matched via keyword -> "task"/"prompt"/"focus")
if any(k in sid for k in ["task", "genre", "purpose", "audience", "prompt", "component"]):
    cc = base["claim_candidate_count"]; pc = base["paragraph_count"]
    if cc >= 3 and pc >= 3: return DEPTH_2
    if cc >= 1: return DEPTH_1
    return DEPTH_0
```

Again, only claim count and paragraph count are checked — structure, not subject matter. A fluent, well-organized essay about the wrong topic sails through both checks at `DEPTH_2` (top score), because well-organized is exactly what it measured.

This is compounded by a second issue: the Evaluator stage is currently run **without its LLM judge**. In `gold_engine_commands_full_v1_4_13.json`, the evaluator command has no `--use-llm` flag (unlike `lret_session`, which does). So every run, not just this one, is scored by the structural heuristic above, with no model in the loop that could actually read the prompt and the essay side by side and notice a mismatch.

Recommendation

- Add a real semantic check: pass `prompt_text` (already accepted as a parameter at line 394 and already plumbed into `extract_text_maps()` at line 4707, it's just never used for relevance) into an LLM call or embedding-similarity check that scores how well the essay's claims actually address the assigned prompt, and use that to gate `arg_claim_relevance` and `maintain_task_focus` instead of "does a claim exist."
- As an interim, cheap guardrail: add a hard ceiling — if a fast keyword/embedding overlap between essay and prompt is below a threshold, cap `task_response` at band 2-3 regardless of what the structural heuristics say. This alone would have caught this case.
- Turn on `--use-llm` for the evaluator stage (see the model-tier note further down) so a model is actually reading the prompt, not just counting claims.

Finding 2 — Canonical lexical resources still failing to load in production

SEVERITY: HIGH — degrades every LRET suggestion in every live session

LRET's own QA warnings for this session state:

```
v168_canonical_resources_silent_load_failure:
--canonical-resources path does not exist on disk: '/app/resources'.
```

This is the old path. Earlier this session your codebase was changed to bake the resource files (lexical_registry.json, positive_collocations_registry.tsv, etc., roughly 74MB, already committed under stella-frontend/app/resources/) into the image and point STELLA_CANONICAL_RESOURCES_DIR at /app/app/resources, the real path they land at inside the container, because the Dockerfile's COPY stella-frontend/ ./ preserves the app/ subfolder. Yet this live run still tried /app/resources.

I confirmed by direct code inspection that this isn't a bug in the LRET script itself or in the command template: lret_engine_v1_12_0...py's own --canonical-resources flag has default=None (no hardcoded fallback baked in), and gold_engine_commands_full_v1_4_13.json's lret_session command correctly passes --canonical-resources {canonical_resources_dir}. That means whatever value the orchestrator resolved for canonical_resources_dir at run time was still the old one.

Two explanations are consistent with what we can see, and they haven't yet been distinguished:

- The container serving this request was still the pre-fix build (a stale or in-flight deploy) rather than the latest image.
- There is an explicit STELLA_CANONICAL_RESOURCES_DIR environment variable set directly in Render's dashboard (Environment tab). A dashboard env var silently overrides a Dockerfile ENV instruction of the same name, so even a correct Dockerfile fix would have no effect if this exists.

Concrete next step: open the Render dashboard, your service, Environment tab, and check whether STELLA_CANONICAL_RESOURCES_DIR is listed there. If it is and it says /app/resources, delete it (or correct it to /app/app/resources) and redeploy. Then submit one more essay and check that lret_session.qa.warnings is empty and resource_support.unit_resource_score is nonzero on at least some units.

This is very likely a major contributor to Finding 3 (fragments and duplicates) below — without the collocation registry loaded, LRET has no grounded phrase database to check candidate spans against, and falls back to weaker heuristics only.

Finding 3 — Same span classified as both FIX and KEEP

SEVERITY: MEDIUM — confusing, contradictory feedback shown to the student simultaneously

In this session's real data, the span "**sound mental**" (a fragment of "sound mental health") appears in both fix_units (flagged as an error to correct) and keep_units (praised as good usage) at the same time. The engine has code specifically designed to prevent this:

```
# build_keep_units(), lret_engine_v1_12_0..., line ~1462
if coverer and coverer.get("class_label") == "FIX":
    unresolved.append({
        "reason": "covered_by_lexical_fix_not_positive_keep", ...
    })
    continue # correctly excluded from keep_units
```

So the intent is right: a KEEP candidate that overlaps a FIX span should be dropped. The bug is in `find_covering_task_for_keep()`, the function that decides whether two spans overlap. For "sound mental" it evidently failed to match the KEEP candidate to the FIX task unit, most likely because the two were extracted as separate n-gram windows (different token boundaries or sentence-index bookkeeping) rather than matched as the same underlying phrase region. Worth checking directly once canonical resources (Finding 2) are confirmed loading, since a missing collocation registry can also change which n-gram windows get proposed as candidates in the first place — it's possible some of this resolves itself once Finding 2 is fixed, and what's left afterward is the true independent bug in the coverage check.

Finding 4 — Lexical unit fragments (partial catch)

SEVERITY: MEDIUM — some fragments slip through the existing filter

LRET does have a noise filter that catches some fragments (units tagged internally as `fragment_or_boundary_artifact` and routed to `dropped_units`, e.g. "private so" was correctly caught this session). But others, like "sound mental" above, are not caught and reach the student-facing output. This points to the same underlying cause as Finding 3: n-gram boundary detection that's inconsistent across the FIX/KEEP/dropped code paths. Recommend re-testing this specifically once Finding 2 is resolved, since a working collocation registry should let the engine recognize "sound mental health" as the real 3-word unit and stop splitting it.

Finding 5 — "Progress tracker is empty"

SEVERITY: MEDIUM — likely a frontend display bug, not a data bug

Checked directly against this session's raw backend output: `progress_tracker`, `progress_tracker_persist`, and `persisted_profile` all contain real, well-formed data — score events, trend-stability flags, and a persisted learner profile were all written correctly for this session.

This contradicts the "empty" experience, which means the gap is very likely on the frontend: whatever page renders progress to the student (a `/progress` or `/study-plan` route) is either not reading this data, reading the wrong session, or has a condition that hides it on a student's first session. This needs to be checked against the actual page component next; it was not located in this pass.

Finding 6 — Writing Coach mission doesn't match Directive's top priority

SEVERITY: MEDIUM — not a bug, but currently unexplainable to a student or you

For this session: the Directive engine's `next_best_skill` is **sentence_control** (grammar, the one real `CLAUSE_STRUCTURE` error found). Writing Coach's `today_mission.target_skill_id` is **paragraph_planning**, via its own internal move selection (`MWM_PARAGRAPH_IDEA_TO_TOPIC_SENTENCE_V1`, primary microskill `topic_sentence_control`).

These are two different engines making two different recommendations from two different internal states, and this is the student's very first session (only one grammar error has ever been logged, and

`practice_session.welcome.targets` is a generic 11-skill bootstrap list, not real differentiated data yet). So the mismatch is expected for session #1, but the underlying issue, that Writing Coach does not consume Directive's `next_best_skill` as an input, will keep producing hard-to-explain picks later too, once there's real signal. Recommend wiring Directive's top-priority skill into Writing Coach's move-selection as a weighted input, or at minimum surfacing "why this mission" to the student, referencing the same skill Directive flagged, so the two don't visibly disagree.

Open decision — stronger model for Detector and Evaluator

Not yet acted on. Currently both `VIP_STRONG_MODEL` (Detector) and `OPENAI_MODEL` (Evaluator) default to `gpt-4o-mini` for every tier — there is no actual weak/strong split happening yet. Given Finding 1 recommends turning on the Evaluator's LLM judge, this is a good moment to also decide whether that judge should run on a stronger model than `gpt-4o-mini`, since it would now be doing the one job (real topic-relevance judgment) that most benefits from a stronger model. Awaiting your go-ahead to change the env vars in Render.

Prioritized next steps

- **1.** Check Render's Environment tab for a stray `STELLA_CANONICAL_RESOURCES_DIR` override (Finding 2) — quick, unblocks re-testing LRET quality honestly.
- **2.** Decide on the Task Response fix approach (Finding 1) — at minimum, ship the cheap embedding/keyword-overlap guardrail so an off-topic essay can never again score above band 2-3, while the fuller LLM-based relevance check is built.
- **3.** Re-run one essay after Finding 2 is fixed and re-check Findings 3 and 4 (FIX/KEEP duplicate, fragments) — some of this may resolve on its own.
- **4.** Locate and check the frontend progress page against Finding 5's confirmed-present backend data.
- **5.** Decide on wiring the Directive to Writing Coach priority signal (Finding 6) and on the stronger-model question.